READING SHEET

Data Representation

Module 3 · Frontend: React & Next.js · Day 46 · Week 10

PROGRAM	WORK TYPE	EST. TIME
CodeOps · Full Stack	**Revision + mini-project**	**2–3 hours**

> **Tip How to use this sheet**
> Week 10 opens with the screens that show data rather than collect it. This sheet covers choosing between a chart and a table, building both so they are readable and honest, paging without losing the URL, and the three states where there is nothing to show — which is most of the work on any data screen.

Part 1 · Choosing a Representation

A **chart** answers a question about shape — is it rising, which is largest, how are they spread. A **table** answers a question about a value. Reaching for the wrong one is the most common mistake in a dashboard.

USE A CHART WHEN	USE A TABLE WHEN
The shape is the point	The number is the point
Comparing a few categories	Someone will look up one row
Showing change over time	Values need to be copied out
Spotting an outlier	There are many columns
The audience is scanning	The audience is checking

"Are orders growing?" wants a line. "What did order 812 cost?" wants a table row. A chart for the trend with a table beneath it for the detail is a good default — scan first, then check.

Which chart

TYPE	ANSWERS	ADDIS EATS EXAMPLE
Line	How has it changed?	Orders per day this month
Bar	Which is largest?	Revenue by neighbourhood
Stacked bar	What makes up the total?	Dishes within each area
Pie	Roughly what share?	Payment method split
Scatter	Are these related?	Delivery time against distance

> **Warning The pie chart warning**
> People compare angles badly — two slices at 22 and 25 per cent look identical, and anything past four slices becomes decoration. A horizontal bar chart answers the same question more accurately and holds far more categories.

Honest charts

DISHONEST	HONEST
A bar axis starting at 400	Bars start at zero, always
Unlabelled axes	Units named — ETB, orders, minutes
Colour as the only key	Labels, or a pattern as well
A truncated time range	The range stated on the chart
Three different scales	One scale, or separate charts

The length of a bar *is* the value, so starting the axis at 400 makes a nine per cent difference look like a threefold one — a line chart may start elsewhere, a bar chart may not. And roughly one man in twelve cannot distinguish red from green: if the only way to read your chart is by hue, some of your audience cannot read it.

Getting the data ready

```
                                                shape it on the server, chart it on the client
// server component
const byArea = areas.map((area) => ({ area,
  revenue: orders.filter((o) => o.area === area)
                 .reduce((sum, o) => sum + o.total, 0) }));

return <RevenueChart data={byArea} />;   // client leaf
```

Send three summarised rows to the browser, not four thousand orders. Charting libraries are client code, so the Day 38 boundary applies here more than anywhere — and the data must be plain arrays and objects, since anything crossing must serialise.

Part 2 · Charts with Recharts

A chart is **composed** from parts rather than configured from an object — axes, tooltip and bars are children, not options. That takes ten minutes to get used to and then makes every customisation obvious.

```
                                                                app/dashboard/RevenueChart.jsx
"use client";   // import from "recharts"
<ResponsiveContainer width="100%" height={300}>
  <BarChart data={data}>
    <XAxis dataKey="area" /><YAxis /><Tooltip />
    <Bar dataKey="revenue" fill="#1B4D8E" />
  </BarChart></ResponsiveContainer>
```

> **Warning It must be a client component, with a real height**
> Charts measure the DOM to size themselves, so keep the page a server component and make only the chart a client leaf. And a percentage height inside a parent with no height collapses to nothing — the first bug everyone hits. Give the wrapper a real height in CSS, or the container a number. A fixed height also reserves the space, so nothing jumps when the chart appears.

Making it readable

```
                                                    formatting, in ETB, with a real label
const etb = (v) => new Intl.NumberFormat("en-ET", {
  style: "currency", currency: "ETB",
  maximumFractionDigits: 0 }).format(v);

<YAxis tickFormatter={etb} width={80} />
<Tooltip formatter={(v) => [etb(v), "Revenue"]} />
```

"14200" means nothing on an axis; "ETB 14,200" means something — one formatter, reused by the axis and the tooltip, with no library. Set an explicit width on the Y axis, because formatted labels are wider than raw ones and will otherwise be clipped.

A chart people can actually use

DO	WHY
Give it a heading above	A chart with no title is a puzzle
State the range and the unit	"Orders per day, June 2026"
Provide the numbers as a table	Screen readers cannot read an SVG
Limit to a few series	Five lines is a graph; ten is a mess
Use your brand palette	Consistency across the dashboard

The honest accessibility fix for a chart is the same data as a table beneath it — which most people find useful anyway, so nothing is wasted. Chart.js is a reasonable alternative: canvas rather than SVG, faster with very large datasets, configured with an object instead of composed.

Part 3 · Tables & Data Grids

A table is the most underrated component in any application. Built from real table markup it is accessible, printable and searchable for free — and a read-only table has no state and no handlers, so it stays a server component and ships nothing to the browser.

```
                                              app/orders/OrderTable.jsx — a server component
<table>
  <caption>Your orders, most recent first</caption>
  <thead><tr><th scope="col">Order</th>
             <th scope="col">Total</th></tr></thead>
  <tbody>{orders.map((o) => (
    <tr key={o.id}><td>{o.id}</td>
                   <td>{etb(o.total)}</td></tr>))}</tbody>
</table>
```

> **Warning Not a grid of divs**
> Divs with CSS grid look identical and are unreadable to a screen reader, uncopyable and unprintable. The scope attribute and the caption are what make it a table rather than a picture of one.

Designing the columns

RULE	WHY
Numbers right-aligned	Digits line up and can be compared
Text left-aligned	The eye follows a straight left edge
Tabular figures for money	Columns of ETB stay in line
Most important column first	It survives a narrow screen
Five or six columns at most	More becomes a spreadsheet

Right-aligned numbers make a column of totals scannable at a glance — the single change that most improves a data table, and it costs one class. On a phone, either scroll the table horizontally in a labelled container or switch to a card per row; never shrink the text until it is unreadable.

Sorting

keep the state in the URL

```
// /orders?sort=total&dir=desc
const orders = await getOrdersFor(session.id, {
  sort: searchParams.sort ?? "date",
  dir:  searchParams.dir  ?? "desc" });
```

The sort lives in the URL, so it is shareable and survives a refresh, and the table stays a server component. Sorting in the browser means shipping every row *and* the sorting code. Set `aria-sort` on the sorted column header — without it a screen reader has no idea the order changed.

When to reach for a grid library

HAND-BUILT IS ENOUGH	A LIBRARY EARNS ITS SIZE
Up to a few hundred rows	Tens of thousands, virtualised
Sorting on the server	Column resize and reorder
A handful of columns	Grouping and pinned columns
Read-only	Editing cells in place
One or two tables	A whole admin application

TanStack Table is the choice when you do need one — it is **headless**, giving you the logic and no markup, so your own accessible table elements stay. For this module and most capstones, the left column is the honest answer.

Part 4 · Paging & the States

	PAGINATION	INFINITE SCROLL
Best for	Records people look up	Feeds people browse
The URL	Linkable — /orders?page=3	Not linkable at all
Going back	Returns to the same page	Loses your position
The footer	Reachable	Effectively unreachable

Infinite scroll feels modern and quietly costs you a shareable URL, a working back button and a reachable footer. Use it where people browse without a destination; use numbered pages wherever someone might want to return to what they found — which for an order history is always.

```
                                                    the page number belongs in the URL
const page = Number(searchParams.page ?? 1);
const { rows, total } = await getOrdersFor(session.id, {
  skip: (page - 1) * 20, take: 20 });

<Pager page={page} pages={Math.ceil(total / 20)} />
```

Twenty rows and a count cross the network, not four thousand. Make each page a real `Link` to a real URL — that gives you the back button, middle-click and a crawler that can follow it, all for free.

The four states

STATE	WHAT TO SHOW
Loading	A skeleton the shape of the content
Empty	Why it is empty, and what to do next
Error	What failed, and a way to retry
Loaded	The data, formatted properly

Three of the four are cases where there is nothing to show, which is why building only the loaded state is what makes applications feel broken on a slow connection. You already have two: `loading.js` and `error.js` from Day 36 cover the segment. The empty state is yours to write, and it is the one people notice — "No data" is a failure; "No orders yet — browse the menu to place your first", with a link, is a screen.

Skeletons that match

```
                                                                    app/orders/loading.js
export default function Loading() {
  return <div className="space-y-2">
    {Array.from({ length: 5 }).map((_, i) => (
      <div key={i} className="h-10 bg-slate-200 animate-pulse" />
    ))}</div>;
}
```

A skeleton shaped like five table rows tells someone what to expect and reserves the space, so nothing jumps when the data lands. A centred spinner tells them only that something is happening.

Assigned Reading

Required

1. **Recharts — getting started** What / why: composition, the responsive container, and the common chart types.
2. **MDN — HTML table advanced features** What / why: scope, caption and thead, and why they matter for assistive technology.

Optional — go deeper

- **NN/g — Data table design** — alignment, column order and readability, from usability research.
- **TanStack Table** — the headless approach, for when a hand-built table stops being enough.

Exercises

Build the empty state first — before there is any data to display.

1. Aggregate order data in a server component and pass only the summary to the client.
2. Build the revenue bar chart with a fixed-height container and ETB tick formatting.
3. Add an orders-per-day line chart with a heading naming the range and the unit.
4. Put the same chart data in a table beneath it for anyone who cannot see the SVG.
5. Build the order table with real markup, right-aligned totals and a caption.
6. Add sorting via searchParams, with aria-sort on the active column header.
7. Add paging with Link elements, then write and force each of the four states.

Mini-Project · The Addis Eats Dashboard

> **Tip What you build**
> A dashboard with revenue by neighbourhood as a bar chart and orders over time as a line, both formatted in ETB and readable without colour; an accessible order table paged and sorted from the URL; and all four states implemented and demonstrable on demand.

Requirements

- Aggregation done on the server, with only summarised rows crossing the boundary.
- Charts as client leaves on a server page, each with a heading, unit and fixed height.
- Every chart accompanied by the same data in a table.
- A table built from table, caption, thead and th scope — not divs.
- Sorting and paging both in the URL, with paging using Link elements.
- Loading, empty and error states all written, and each one demonstrable.

Check yourself

- Does every bar chart axis start at zero?
- Can you read every chart with the colour removed?
- How many rows cross the network for the dashboard — three, or four thousand?
- Does the table survive being copied into a spreadsheet?
- Can you link someone directly to page three sorted by total?
- What does a brand-new customer see on the orders page?